

3_feature_engineering_exp

April 13, 2026

1 Adaptive Guardrail Layer (AGL) - Feature Engineering Notebook

1.1 Purpose

This notebook creates engineered features from the cleaned prompt dataset for two downstream use cases:

1. **Binary classification** of malicious vs benign prompts
2. **Unsupervised anomaly detection** to identify unusual or outlier prompts

1.2 Design Goals

The engineered features are intended to complement raw prompt text rather than replace it.

For our deep learning model, the raw prompt remains the primary signal.

For our anomaly detection, compact numeric and semantic features help represent prompt behavior in a structured way.

1.3 Input

- dataset_cleaned.csv

1.4 Output

- dataset_feature_engineered.csv

```
[2]: import os
import re
import string
from pathlib import Path
import numpy as np
import pandas as pd
from transformers import AutoTokenizer
from sentence_transformers import SentenceTransformer
from sklearn.decomposition import PCA
from sklearn.preprocessing import LabelEncoder

pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", 300)
```

1.4.1 Define file paths

This notebook loads the cleaned dataset from the processed data folder and writes one final engineered dataset back to the same folder.

```
[4]: INPUT_PATH = "../../../data/processed/"
      OUTPUT_PATH = "../../../data/processed/"

      input_file = Path(INPUT_PATH) / "dataset_cleaned.csv"
      output_file = Path(OUTPUT_PATH) / "dataset_feature_engineered.csv"

      if not os.path.exists(input_file):
          raise FileNotFoundError(f"Could not find input file: {input_file}")

      print(f"Input file found: {input_file}")
```

Input file found: ../../../data/processed/dataset_cleaned.csv

1.4.2 Load the cleaned dataset

We load the cleaned dataset and create a stable row identifier so records can always be traced during modeling and analysis.

```
[6]: df = pd.read_csv(input_file)

      df = df.reset_index(drop=True).copy()
      df["row_id"] = df.index

      print("Dataset shape:", df.shape)
      display(df.head())
```

Dataset shape: (104755, 4)

```

prompt \
0 Here's what I need you to do for me: 1. Translate 'Good morning' to French. 2.
  Calculate the area of a circle with a radius of 5cm. 3. Provide the capital
  of Japan. 4. Give an example of a plant that lives in the desert.
1 I need some information quickly: 1. What is
  the boiling point of water? 2. Who wrote 'To Kill a Mockingbird'? 3. Find the
  square root of 144. 4. What is the currency of the United Kingdom?
2 Please do the
  following tasks: 1. Explain what SEO stands for. 2. List the primary colors. 3.
  What year did the Titanic sink? 4. How many continents are there?
3 Could you help with these items? 1. Define
  'Machine Learning'. 2. What is the chemical symbol for Gold? 3. How do you say
  'thank you' in Spanish? 4. Name a country that starts with 'D'.
```

4 I'm curious
 about a few things: 1. What's the distance to the Moon? 2. Name a book by Jane
 Austen. 3. Convert 100°F to Celsius. 4. What does HTTP stand for?

	label	source_dataset	row_id
0	0	WildGuardMix	0
1	0	WildGuardMix	1
2	0	WildGuardMix	2
3	0	WildGuardMix	3
4	0	WildGuardMix	4

1.4.3 Validate expected columns

Before creating features, verify that the dataset contains the required columns.

```
[8]: required_columns = {"prompt", "label", "source_dataset"}
     missing_columns = required_columns - set(df.columns)

     if missing_columns:
         raise ValueError(f"Missing required columns: {missing_columns}")

     print("All required columns are present.")
```

All required columns are present.

1.4.4 Standardize prompt text for feature extraction

The raw prompt text is preserved, but we also create a normalized text version for deterministic feature extraction.

This normalized version is used for: - token counting - pattern detection - embedding generation

```
[10]: def normalize_prompt(text: str) -> str:
        text = str(text)
        text = text.replace("\r\n", "\n").replace("\r", "\n")
        text = text.strip()
        return text

     df["prompt_normalized"] = df["prompt"].apply(normalize_prompt)

     display(df[["prompt", "prompt_normalized"]].head())
```

	prompt	prompt_normalized
0	Here's what I need you to do for me: 1. Translate 'Good morning' to French. 2. Calculate the area of a circle with a radius of 5cm. 3. Provide the capital of Japan. 4. Give an example of a plant that lives in the desert.	Here's what I need you to do for me: 1. Translate 'Good morning' to French. 2. Calculate the area of a circle with a radius of 5cm. 3. Provide the capital of Japan. 4. Give an example of a plant that lives in the desert.

```

1 I need some information quickly: 1. What is
↳the boiling point of water? 2. Who wrote 'To Kill a Mockingbird'? 3. Find the
↳square root of 144. 4. What is the currency of the United Kingdom?
2 Please do the
↳following tasks: 1. Explain what SEO stands for. 2. List the primary colors. 3.
↳What year did the Titanic sink? 4. How many continents are there?
3 Could you help with these items? 1. Define
↳'Machine Learning'. 2. What is the chemical symbol for Gold? 3. How do you say
↳'thank you' in Spanish? 4. Name a country that starts with 'D'.
4 I'm curious
↳about a few things: 1. What's the distance to the Moon? 2. Name a book by Jane
↳Austen. 3. Convert 100°F to Celsius. 4. What does HTTP stand for?

↳
↳
↳ prompt_normalized
0 Here's what I need you to do for me: 1. Translate 'Good morning' to French. 2.
↳Calculate the area of a circle with a radius of 5cm. 3. Provide the capital
↳of Japan. 4. Give an example of a plant that lives in the desert.
1 I need some information quickly: 1. What is
↳the boiling point of water? 2. Who wrote 'To Kill a Mockingbird'? 3. Find the
↳square root of 144. 4. What is the currency of the United Kingdom?
2 Please do the
↳following tasks: 1. Explain what SEO stands for. 2. List the primary colors. 3.
↳What year did the Titanic sink? 4. How many continents are there?
3 Could you help with these items? 1. Define
↳'Machine Learning'. 2. What is the chemical symbol for Gold? 3. How do you say
↳'thank you' in Spanish? 4. Name a country that starts with 'D'.
4 I'm curious
↳about a few things: 1. What's the distance to the Moon? 2. Name a book by Jane
↳Austen. 3. Convert 100°F to Celsius. 4. What does HTTP stand for?

```

1.4.5 Create core text length features

These features capture prompt size and are useful for both supervised and unsupervised models. Created features: - char_length - word_count - avg_word_length - line_count - sentence_count

```

[12]: df["char_length"] = df["prompt_normalized"].str.len()
df["word_count"] = df["prompt_normalized"].str.split().str.len()

def avg_word_length(text: str) -> float:
    words = str(text).split()
    if not words:
        return 0.0
    return sum(len(word) for word in words) / len(words)

```

```

df["avg_word_length"] = df["prompt_normalized"].apply(avg_word_length)
df["line_count"] = df["prompt_normalized"].str.count(r"\n") + 1
df["sentence_count"] = df["prompt_normalized"].str.count(r"[.!?]")

display(
    df[
        ["char_length", "word_count", "avg_word_length", "line_count",
         ↪ "sentence_count"]
    ].head()
)

```

	char_length	word_count	avg_word_length	line_count	sentence_count
0	220	45	3.911111	1	8
1	188	36	4.250000	1	8
2	159	29	4.517241	1	8
3	185	35	4.314286	1	9
4	156	31	4.064516	1	8

1.4.6 Create token-based features

Large language models operate on tokens rather than words, so token-based features provide a more model-relevant measure of prompt complexity.

Created features: - token_count - token_word_ratio - token_char_ratio - over_128_tokens - over_256_tokens - over_512_tokens

```

[14]: tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

def count_tokens(text: str) -> int:
    tokens = tokenizer.tokenize(str(text))
    return len(tokens)

df["token_count"] = df["prompt_normalized"].apply(count_tokens)
df["token_word_ratio"] = df["token_count"] / df["word_count"].replace(0, 1)
df["token_char_ratio"] = df["token_count"] / df["char_length"].replace(0, 1)

df["over_128_tokens"] = (df["token_count"] > 128).astype(int)
df["over_256_tokens"] = (df["token_count"] > 256).astype(int)
df["over_512_tokens"] = (df["token_count"] > 512).astype(int)

display(
    df[
        [
            "token_count",
            "token_word_ratio",
            "token_char_ratio",
            "over_128_tokens",
            "over_256_tokens",

```

```

        "over_512_tokens",
    ]
    ].head()
)

```

Token indices sequence length is longer than the specified maximum sequence length for this model (978 > 512). Running this sequence through the model will result in indexing errors

	token_count	token_word_ratio	token_char_ratio	over_128_tokens	\
0	59	1.311111	0.268182	0	
1	48	1.333333	0.255319	0	
2	38	1.310345	0.238994	0	
3	50	1.428571	0.270270	0	
4	48	1.548387	0.307692	0	

	over_256_tokens	over_512_tokens
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0

1.4.7 Create character composition features

These features describe how the prompt is composed and can help detect unusual formatting, obfuscation, or non-standard input patterns.

Created features: - uppercase_ratio - digit_ratio - whitespace_ratio - punctuation_ratio - special_char_ratio - non_ascii_ratio

```

[16]: def safe_len(text: str) -> int:
        return max(len(str(text)), 1)

    def uppercase_ratio(text: str) -> float:
        text = str(text)
        return sum(1 for c in text if c.isupper()) / safe_len(text)

    def digit_ratio(text: str) -> float:
        text = str(text)
        return sum(1 for c in text if c.isdigit()) / safe_len(text)

    def whitespace_ratio(text: str) -> float:
        text = str(text)
        return sum(1 for c in text if c.isspace()) / safe_len(text)

    def punctuation_ratio(text: str) -> float:
        text = str(text)
        return sum(1 for c in text if c in string.punctuation) / safe_len(text)

```

```

def special_char_ratio(text: str) -> float:
    text = str(text)
    return sum(1 for c in text if not c.isalnum() and not c.isspace()) / \
        safe_len(text)

def non_ascii_ratio(text: str) -> float:
    text = str(text)
    return sum(1 for c in text if ord(c) > 127) / safe_len(text)

df["uppercase_ratio"] = df["prompt_normalized"].apply(uppercase_ratio)
df["digit_ratio"] = df["prompt_normalized"].apply(digit_ratio)
df["whitespace_ratio"] = df["prompt_normalized"].apply(whitespace_ratio)
df["punctuation_ratio"] = df["prompt_normalized"].apply(punctuation_ratio)
df["special_char_ratio"] = df["prompt_normalized"].apply(special_char_ratio)
df["non_ascii_ratio"] = df["prompt_normalized"].apply(non_ascii_ratio)

display(
    df[
        [
            "uppercase_ratio",
            "digit_ratio",
            "whitespace_ratio",
            "punctuation_ratio",
            "special_char_ratio",
            "non_ascii_ratio",
        ]
    ].head()
)

```

	uppercase_ratio	digit_ratio	whitespace_ratio	punctuation_ratio \
0	0.040909	0.022727	0.200000	0.054545
1	0.053191	0.037234	0.186170	0.058511
2	0.056604	0.025157	0.176101	0.056604
3	0.054054	0.021622	0.183784	0.081081
4	0.089744	0.044872	0.192308	0.070513

	special_char_ratio	non_ascii_ratio
0	0.054545	0.00000
1	0.058511	0.00000
2	0.056604	0.00000
3	0.081081	0.00000
4	0.076923	0.00641

1.4.8 Create formatting and structure features

Adversarial prompts often contain strong formatting structure, repeated punctuation, markdown artifacts, or code-like content.

Created features: - newline_count - markdown_symbol_count - has_code_block - repeated_punct_count - quote_count - bracket_count - colon_count - semicolon_count

```
[18]: df["newline_count"] = df["prompt_normalized"].str.count(r"\n")
df["markdown_symbol_count"] = df["prompt_normalized"].str.count(r"`*_#>-")
df["has_code_block"] = df["prompt_normalized"].str.contains(r"```", regex=True).
    ↪astype(int)

df["repeated_punct_count"] = df["prompt_normalized"].apply(
    lambda x: len(re.findall(r"([!?.,:;])\1{1,}", str(x)))
)

df["quote_count"] = df["prompt_normalized"].str.count(r"\"'\")
df["bracket_count"] = df["prompt_normalized"].str.count(r"[\(\)\[\]\{\}"]
df["colon_count"] = df["prompt_normalized"].str.count(":")
df["semicolon_count"] = df["prompt_normalized"].str.count(";")

display(
    df[
        [
            "newline_count",
            "markdown_symbol_count",
            "has_code_block",
            "repeated_punct_count",
            "quote_count",
            "bracket_count",
            "colon_count",
            "semicolon_count",
        ]
    ].head()
)
```

	newline_count	markdown_symbol_count	has_code_block	repeated_punct_count	\
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0

	quote_count	bracket_count	colon_count	semicolon_count
0	3	0	1	0
1	2	0	1	0
2	0	0	1	0
3	6	0	0	0
4	2	0	1	0

1.4.9 Create heuristic security-pattern features

These are interpretable rule-based features that capture common adversarial prompt behaviors.

The goal is not to replace the model's understanding, but to add lightweight signals that may help:

- binary classification - anomaly detection - post-hoc analysis

Created groups: - instruction override patterns - roleplay / jailbreak patterns - payload request patterns - social engineering patterns - obfuscation patterns

```
[20]: instruction_override_phrases = [
    "ignore previous instructions",
    "ignore all previous instructions",
    "disregard previous instructions",
    "forget your instructions",
    "system override",
    "developer mode",
    "override your rules",
    "ignore all prior"
]

roleplay_phrases = [
    "act as",
    "pretend to be",
    "roleplay as",
    "you are now",
    "unrestricted ai",
    "without rules",
    "evil assistant",
    "dan"
]

payload_phrases = [
    "write malware",
    "create ransomware",
    "sql injection",
    "ddos attack",
    "exploit code",
    "reverse shell",
    "shellcode",
    "keylogger",
    "payload",
    "phishing page"
]

social_engineering_phrases = [
    "phishing email",
    "scam message",
    "impersonate",
```

```

        "trick someone",
        "social engineer",
        "convince a user",
        "spoof email",
        "fraud message"
    ]

    obfuscation_phrases = [
        "base64",
        "rot13",
        "leet",
        "hex encoded",
        "encoded payload",
        "character by character",
        "spaced letters",
        "obfuscate"
    ]

    def phrase_score(text, phrase_list):
        text = str(text).lower()
        return sum(1 for phrase in phrase_list if phrase in text)

    df["instruction_override_score"] = df["prompt_normalized"].apply(
        lambda x: phrase_score(x, instruction_override_phrases)
    )
    df["roleplay_score"] = df["prompt_normalized"].apply(
        lambda x: phrase_score(x, roleplay_phrases)
    )
    df["payload_request_score"] = df["prompt_normalized"].apply(
        lambda x: phrase_score(x, payload_phrases)
    )
    df["social_engineering_score"] = df["prompt_normalized"].apply(
        lambda x: phrase_score(x, social_engineering_phrases)
    )
    df["obfuscation_score"] = df["prompt_normalized"].apply(
        lambda x: phrase_score(x, obfuscation_phrases)
    )

    df["has_instruction_override"] = (df["instruction_override_score"] > 0).
        astype(int)
    df["has_roleplay"] = (df["roleplay_score"] > 0).astype(int)
    df["has_payload_request"] = (df["payload_request_score"] > 0).astype(int)
    df["has_social_engineering"] = (df["social_engineering_score"] > 0).astype(int)
    df["has_obfuscation"] = (df["obfuscation_score"] > 0).astype(int)

    display(
        df[

```

```

[
    "instruction_override_score",
    "roleplay_score",
    "payload_request_score",
    "social_engineering_score",
    "obfuscation_score",
    "has_instruction_override",
    "has_roleplay",
    "has_payload_request",
    "has_social_engineering",
    "has_obfuscation",
]
].head()
)

```

	instruction_override_score	roleplay_score	payload_request_score	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	social_engineering_score	obfuscation_score	has_instruction_override	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	has_roleplay	has_payload_request	has_social_engineering	has_obfuscation
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0
4	0	0	0	0

1.4.10 Encode the source dataset

The source dataset is encoded as a categorical feature so downstream models can capture dataset-specific patterns or bias.

This can help both: - supervised models - anomaly detection models

```

[22]: source_encoder = LabelEncoder()
df["source_dataset_encoded"] = source_encoder.
    ↪fit_transform(df["source_dataset"].astype(str))

display(

```

```
df[["source_dataset", "source_dataset_encoded"]]
    .drop_duplicates()
    .sort_values("source_dataset")
)
```

	source_dataset	source_dataset_encoded
54605	MPDD	0
49526	Malicious LLM Prompts	1
93712	Prompt Injection & Benign Prompt Dataset	2
0	WildGuardMix	3
102807	deepset/prompt-injections	4
103469	jackhhao/jailbreak-classification	5
94210	malicious-llm-prompts-v4	6

1.4.11 Generate semantic embeddings

Embeddings provide a compact semantic representation of each prompt.

- For binary classification: embeddings help capture meaning beyond exact keywords
- For anomaly detection: embeddings make it easier to detect prompts that are semantically unusual

Because raw embeddings are high dimensional, we will reduce them with PCA before saving to CSV.

```
[ ]: embedding_model = SentenceTransformer("all-MiniLM-L6-v2")

texts = df["prompt_normalized"].astype(str).tolist()

embeddings = embedding_model.encode(
    texts,
    batch_size=64,
    show_progress_bar=True,
    convert_to_numpy=True
)

print("Raw embedding shape:", embeddings.shape)
```

huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid deadlocks...

To disable this warning, you can either:

- Avoid using `tokenizers` before the fork if possible
- Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

false)

Batches: 0% | 0/1637 [00:00<?, ?it/s]

1.4.12 Create compact embedding-derived features

In addition to PCA-reduced embedding components, we compute a few lightweight summary features from the embedding vectors.

Created features: - embedding_dimension - embedding_norm

```
[ ]: df["embedding_dimension"] = embeddings.shape[1]
df["embedding_norm"] = np.linalg.norm(embeddings, axis=1)

display(df[["embedding_dimension", "embedding_norm"]].head())
```

1.4.13 Reduce embeddings with PCA

To keep the final dataset manageable and CSV-friendly, we reduce the embedding vectors to a smaller number of principal components.

This notebook uses 32 PCA components as a balanced default: - small enough for CSV output - large enough to preserve useful semantic information

```
[ ]: N_COMPONENTS = 32

pca = PCA(n_components=N_COMPONENTS, random_state=42)
embeddings_pca = pca.fit_transform(embeddings)

embedding_cols = [f"embed_pca_{i+1}" for i in range(N_COMPONENTS)]
embeddings_pca_df = pd.DataFrame(embeddings_pca, columns=embedding_cols)

print("Reduced embedding shape:", embeddings_pca_df.shape)
print("Explained variance ratio sum:", pca.explained_variance_ratio_.sum())

display(embeddings_pca_df.head())
```

1.4.14 Combine all engineered features into one final dataset

The final dataset keeps the original core fields and appends the engineered numeric features.

```
[ ]: engineered_feature_cols = [
    "row_id",
    "char_length",
    "word_count",
    "avg_word_length",
    "line_count",
    "sentence_count",
    "token_count",
    "token_word_ratio",
    "token_char_ratio",
    "over_128_tokens",
    "over_256_tokens",
    "over_512_tokens",
    "uppercase_ratio",
    "digit_ratio",
    "whitespace_ratio",
    "punctuation_ratio",
```

```

    "special_char_ratio",
    "non_ascii_ratio",
    "newline_count",
    "markdown_symbol_count",
    "has_code_block",
    "repeated_punct_count",
    "quote_count",
    "bracket_count",
    "colon_count",
    "semicolon_count",
    "instruction_override_score",
    "roleplay_score",
    "payload_request_score",
    "social_engineering_score",
    "obfuscation_score",
    "has_instruction_override",
    "has_roleplay",
    "has_payload_request",
    "has_social_engineering",
    "has_obfuscation",
    "source_dataset_encoded",
    "embedding_dimension",
    "embedding_norm",
]

df_final = pd.concat(
    [
        df[["row_id", "prompt", "label", "source_dataset"]].copy(),
        df[engineered_feature_cols].drop(columns=["row_id"]).copy(),
        embeddings_pca_df.reset_index(drop=True),
    ],
    axis=1
)

print("Final engineered dataset shape:", df_final.shape)
display(df_final.head())

```

1.4.15 Save the final engineered dataset

```

[ ]: df_final.to_csv(output_file, index=False)

print(f"Feature-engineered dataset saved to: {output_file}")
print("Final shape:", df_final.shape)

```

2 Feature Summary Table

Feature Name	Values	Description
prompt	string	Original user input text
label	{0,1}	Binary target label (1 = malicious, 0 = benign)
source_dataset	categorical (string)	Original dataset source
source_dataset_encoded	integer	Label-encoded dataset source
attack_label	categorical	LLM-predicted attack category (Jailbreaking, Prompt Injection, Malicious Payload Generation, Social Engineering, Token Smuggling/Obfuscation, Benign, Other Malicious)
char_length	integer (≥ 0)	Number of characters in the prompt
word_count	integer (≥ 0)	Number of words in the prompt
avg_word_length	float (≥ 0)	Average length of words
line_count	integer (≥ 1)	Number of lines in the prompt
sentence_count	integer (≥ 0)	Approximate number of sentences
token_count	integer (≥ 0)	Number of tokenizer tokens
token_word_ratio	float	Tokens divided by word count
token_char_ratio	float	Tokens divided by character count
uppercase_ratio	float [0–1]	Proportion of uppercase characters
digit_ratio	float [0–1]	Proportion of numeric characters
whitespace_ratio	float [0–1]	Proportion of whitespace characters
punctuation_ratio	float [0–1]	Proportion of punctuation characters
special_char_ratio	float [0–1]	Proportion of non-alphanumeric characters
non_ascii_ratio	float [0–1]	Proportion of non-ASCII characters
newline_count	integer (≥ 0)	Number of newline characters
markdown_symbol_count	integer (≥ 0)	Count of markdown-style symbols (*, _, #, etc.)
has_code_block	{0,1}	Indicates presence of triple backtick code blocks
repeated_punct_count	integer (≥ 0)	Count of repeated punctuation patterns (e.g., !!!, ???)
quote_count	integer (≥ 0)	Number of quotation marks
bracket_count	integer (≥ 0)	Number of brackets ((), [], {})
instruction_override_score	integer (≥ 0)	Count of phrases attempting to override instructions
roleplay_score	integer (≥ 0)	Count of roleplay/jailbreak phrases
payload_request_score	integer (≥ 0)	Count of malicious payload-related terms
social_engineering_score	integer (≥ 0)	Count of social engineering-related phrases

Feature Name	Values	Description
obfuscation_score	integer (≥ 0)	Count of encoding/obfuscation indicators
has_instruction_override	{0,1}	Whether instruction override phrases are present
has_roleplay	{0,1}	Whether roleplay patterns are present
has_payload_request	{0,1}	Whether payload request patterns are present
has_social_engineering	{0,1}	Whether social engineering patterns are present
has_obfuscation	{0,1}	Whether obfuscation patterns are present